# CS492 Senior Design Project II

# Detailed Design Report


Elif Ercan 22201601

Bilge İdil Öziş 22102365

Kağan Rehber 22101777

Mennatallah Abouelenin 22101539

Bilgehan Tuğcu 22202667

**1. Introduction** 3
1.1 Purpose of the system 3
1.2 Design goals 3
1.2.1 Usability 3
1.2.2 Reliability 3
1.2.3 Performance 4
1.2.4 Supportability 4
1.2.5 Scalability 4
1.2.6 Marketability 4
1.2.7 Extendibility 5
1.2.8 Security 5
1.2.9 Maintainability 5
1.2.10 Flexibility 5
1.2.11 Modularity 6
1.2.12 Aesthetics 6
1.3 Definitions, Acronyms, and Abbreviations 6
1.4 Overview 7
**2. Current Software Architecture** 7
2.1 Overview 7
2.2 Competitors 8
2.2.1 Forest 8
2.2.2 Focus To-Do 8
2.2.3 Fabulous 8
2.2.4 RescueTime and Freedom 8
**3. Proposed Software Architecture** 9
3.1 Overview 9
3.2 Subsystem Decomposition 11
3.2.1 Presentation Layer 11
3.2.2 Service Layer 12
3.2.3 AI Processing Layer 12
3.2.4 Data Storage Layer 13
3.2.5 Native Platform Layer 13
Layer-to-Layer Communication 13
Advantages of the Architecture 14
3.3 Hardware/Software Mapping 15
3.4 Persistent Data Management 15
3.5 Access Control and Security 15
**4. Subsystem Services** 16
4.1 Client Subsystem 16
4.2 Backend Subsystem 16
4.3 AI Subsystem 17
4.4 Object and Class Model 17
4.4.1 User Domain 18

4.4.2 Companion Domain..... 19
4.4.3 Task Domain..... 19
4.4.4 Focus Domain..... 20
4.4.5 Notes Domain..... 20
4.4.6 Schedule Domain..... 21
4.4.7 Mood-tracking and Statistics Domain..... 21
**5. Test Cases..... 22**
**6. Consideration of Various Factors in Engineering Design..... 33**
6.1 Constraints..... 33
6.1.1 Public Health..... 33
6.1.2 Public Safety..... 34
6.1.3 Public Security..... 34
6.1.4 Public Welfare..... 34
6.1.5 Global Factors..... 34
6.1.6 Cultural Factors..... 35
6.1.7 Social Factors..... 35
6.1.8 Environmental Factors..... 35
6.1.9 Economic Factors..... 35
6.2 Standards..... 36
6.2.1 IEEE 830 Software Requirements Specification..... 36
6.2.2 UML 2.5.1..... 37
6.2.3 ISO/IEC 25010..... 37
6.2.4 ISO/IEC/IEEE 12207..... 37
6.2.5 PEP 8..... 37
**7. Teamwork Details..... 37**
7.1 Contributing and functioning effectively on the team..... 37
7.2 Helping creating a collaborative and inclusive environment..... 37
7.3 Taking lead role and sharing leadership on the team..... 38
**8. Glossary..... 38**
**9. References..... 40**

# 1. Introduction

## 1.1 Purpose of the system

The purpose of Sproutly is to offer a comprehensive mobile productivity tool that assists users in dealing with distractions, organizing tasks, and developing good habits of focus in a nurturing environment. It offers a range of features including a task management tool, an AI-based scheduling tool, a focus timer, and gamification features in one platform. It is different from other tools in the market because it does not only offer a timer or a tool that blocks distractions but also assists users in reducing cognitive overload through planning, prioritization, and reflection.

The features of Sproutly include a virtual companion and reward systems to offer support and motivation to users as they complete tasks and develop good habits of focus. It offers AI-based task analysis and suggestions for focus time through a personalized environment. It aims at developing a tool that offers a balance of ethics in design with user empowerment without forcing coercive control on the user.

## 1.2 Design goals

### 1.2.1 Usability

- The application should offer an intuitive, clean, and easy-to-use interface that can be used daily.
- The user experience should remain consistent across iOS and Android platforms, ensuring that users can use the app easily on any device while respecting each platform's native patterns.
- All core features, like starting a focus session, adding tasks, or checking stats, should be within quick access and require a few steps.
- The interface should stay visually clear and pleasant, with readable fonts, suitable text sizes, and relaxing colors for concentration.
- The design should be smooth and reduce cognitive load by excluding unnecessary elements and presenting information in an organized, simple manner.
- The app should provide helpful onboarding so that new users can understand coaching modes, tasks, the companion, and other core features quickly.

### 1.2.2 Reliability

- The app should function predictably and consistently, especially during focus sessions and task tracking.
- User progress, streaks, and statistics should remain accurate and consistent across all app activities.
- The app should remain stable without unexpected crashes and handle errors gracefully without interrupting core workflows.

### 1.2.3 Performance

- The app should run smoothly on standard iOS and Android devices without lag.
- Key features (focus timer, task creation, note-taking, mood check-ins) should load quickly to avoid disrupting user workflow.
- AI-based features such as weekly reports or task suggestions should return results within a reasonable time to maintain a good user experience.
- Animations (such as the companion or transitions) should feel responsive and not significantly impact performance.
- Background syncing operations should not slow down overall app usage.

### 1.2.4 Supportability

- The system should include error handling and logging mechanisms to enable efficient issue detection and troubleshooting during runtime.
- The backend should allow updates and improvements without requiring major changes to the mobile application.
- The app should include clear API documentation to support future integration or external services.
- The design should allow smooth integration of future technologies while keeping the existing system stable.

### 1.2.5 Scalability

- The system should be designed to handle a growing number of users without significant performance decline.
- The data structure should support increasing volumes of notes, tasks, schedules, and focus logs over time.
- The system should support multi-device access, ensuring user data remains consistent across phones and tablets as the number of devices and users grows.
- AI-based features should be designed so they can be improved or expanded in the future as the model evolves.
- The overall architecture should allow adding new coaching modes, companion features, or analytics tools without needing major redesigns.

### 1.2.6 Marketability

- The application should offer a value proposition by offering a combination of AI assistance, focus assistance, and gamification within a single application.
- The application's features should differentiate it from other timer or blocking application options by focusing on personalization and coaching.
- The application's onboarding experience should educate users on the benefits of using the application and its unique features (such as AI breakdown, companion system, proof of work) to ensure that users are not discouraged from continuing to use the application.

- The system's architecture should allow for potential monetization strategies (e.g., premium AI features, cosmetic companion upgrades) without requiring major changes to the system.

### 1.2.7 Extendibility

- The system should allow new features (e.g., additional coaching modes, new AI tools, expanded companion animations) to be introduced without making significant changes to existing components.
- The system's backend API should allow new endpoints to be introduced without affecting existing functionality.
- The database schema should allow new analytics metrics or user attributes to be introduced.

### 1.2.8 Security

- User authentication mechanisms should follow secure industry practices to protect user credentials and session integrity.
- Sensitive user data, such as account information or productivity logs, should be encrypted during transmission and stored securely in the cloud database.
- API endpoints should include proper authorization checks to prevent unauthorized data access.
- The amount of user data collected should be minimal and only sufficient for the application, based on ethics and constraints of user privacy.

### 1.2.9 Maintainability

- The system should be easy to maintain and update without making significant changes in its structure.
- The code should have proper naming conventions and follow a coding style.
- The code should be divided into sections (UI, API, AI module) to avoid confusion during future development.
- The code should have proper comments and documentation to help understand the core components of the code.
- Automated tests should be implemented in the code for the core functionalities of the application to decrease errors while updating.

### 1.2.10 Flexibility

- The system should allow for different user preferences. For example, it should allow users to change the coaching strictness and the focus duration.
- The system should allow users to override the AI suggestions if they wish.
- The UI should be flexible on different screen sizes and follow the design guidelines of the respective platform.
- The system should be able to switch between strict app locking in Android and reminder-based restrictions in iOS without changing the core logic of the system.

### 1.2.11 Modularity

- The system should be divided into independent modules such as Task Management, AI Module, Companion System, Focus Timer, and App Restriction Module.
- Each module should communicate through well-defined interfaces to reduce tight coupling.
- Changes in one module (e.g., updating AI prompts) should not directly impact unrelated modules (e.g., UI rendering).
- The modular structure should allow selective testing and independent development of components.

### 1.2.12 Aesthetics

- The application should present a visually cohesive and calming design aligned with productivity and focus goals.
- The companion animations and milestone visuals should enhance motivation without becoming visually overwhelming.
- Color schemes, typography, and spacing should support readability and long-term daily use.
- Visual feedback (animations, streak celebrations, progress indicators) should reinforce achievement while maintaining a professional appearance.

## 1.3 Definitions, Acronyms, and Abbreviations

**Sproutly:** Name of the application in development. May be generically referred to as ‘app’ or ‘application’ throughout the document.
**OS:** Operating System.
**UI/UX:** User Interface / User Experience. Refers to elements of the app which the user interacts with, and how the app responds to user input.
**API:** Application Programming Interface. A software tool that allows the app to communicate with other third party applications not directly integrated into the app.
**LLM:** Large Language Model. A type of general purpose artificial intelligence good at interpreting natural language and certain other human media.
AI: Artificial Intelligence.
**IEEE:** Institute of Electrical and Electronics Engineers. An institute responsible for setting standards for numerous design, architecture and implementation level concepts for software engineering and other related engineering fields.
**OCR:** Optical Character Recognition.
**KVKK:** Kişisel Verilerin Korunması Kanunu. Turkey’s personal data protection law.
**HIPAA:** Health Insurance Portability and Accountability Act. A US law that protects health related information.
**GDPR:** General Data Protection Regulation. A data privacy regulation of the European Union.

## 1.4 Overview

Sproutly is a mobile application that aims to help users keep track of their habits, improve their productivity, and plan their days in a centralized application that is both easy to use and to learn. Other applications with similar target audiences may be limited in their set of features, lack customizability, or may not adapt dynamically to user behavior.

Sproutly innovates in this space by offering a much broader set of features than its competitors. Weekly schedule planning, tasks & task reminders, systems to reduce phone distractions, customizable focus timers and gamification elements in the form of companions and streaks are all included in one centralized application. The incorporation of AI into certain features helps expand functionality, as well as help take care of cognitive load, such as decision paralysis, which users may face when using productivity apps. The app leverages AI to give feedback and make recommendations based on past user performance and user-provided input.

The application uses Flutter/Dart for frontend, FastAPI for backend, SQLite for local data storage, and Firebase for remote storage solutions. Third-party AI APIs are used for certain AI-assisted features, and the AI API provider being utilized may change depending on pricing, performance, availability, privacy concerns or other relevant factors. End-to-end encryption is used for communication between the client and remote server, where needed.

The aforementioned design decisions come together to ensure the application has a robust foundation that is able to implement and serve a broad range of features.

# 2. Current Software Architecture

## 2.1 Overview

The market for productivity, focus, and habit-tracking applications has expanded significantly in recent years, driven by the widespread adoption of smartphones and the normalization of app-based self-improvement tools. Market reports estimate that the global habit-tracking sector will surpass USD 21 billion by 2031 [1], while the Pomodoro and focus-timer sub-market is projected to grow from USD 320–412 million in 2024 to nearly USD 1 billion by 2033 [2]. This growth is further accelerated by the increasing prevalence of remote and hybrid work, which has amplified digital distractions and the need for tools that help users regulate their attention and maintain productivity [3], [4]. Traditional focus apps, such as Forest or Focus To-Do, address distraction through timers, gamification, or basic task lists, while behavioral-science-based platforms like Fabulous provide motivational routines and structured journeys [5], [6], [7]. Usage-blocking applications like RescueTime or Freedom focus on restricting or monitoring device activity instead of guiding behavior [8], [9]. Although each of these tools supports productivity in different ways, they often rely on rigid mechanisms, lack personalization, or fail to support users in breaking down complex, long-term tasks. Sproutly operates at the convergence of these trends. It addresses the

growing demand for systems that combine productivity support, adaptive guidance, and digital self-control features without the coercive elements or heavy behavioral commitments found in many existing apps. Unlike apps that focus narrowly on timers or blocking, Sproutly integrates motivational feedback, soft-lock focus support, and AI-driven task management to provide a more flexible and supportive experience. With its emphasis on optional accountability and intelligent task breakdown, Sproutly aims to meet the needs of students, freelancers, and knowledge workers seeking a balanced alternative to traditional timer-dominant tools. The detailed analysis of the current competitor ecosystem is as follows:

## 2.2 Competitors

### 2.2.1 Forest

- Forest uses a popular gamified tree-growing mechanic that rewards users for staying off their phones during focus sessions [6].
- While engaging, Forest does not support complex tasks, long-term planning, or adaptive prioritization.
- The app lacks personalized feedback or reflective insights, limiting its usefulness beyond simple focus periods [6].

### 2.2.2 Focus To-Do

- Focus To-Do integrates a Pomodoro timer with a basic task list, allowing users to link tasks with focus sessions [7].
- Despite this combination, the app does not offer intelligent task breakdown, personalized guidance, or contextual suggestions.
- Its functionality remains mostly static, geared toward users with clearly defined tasks rather than those struggling with overwhelming or large projects.

### 2.2.3 Fabulous

- Fabulous is a behavioral-coaching app grounded in behavioral science, offering structured daily routines, motivational content, and goal-oriented journeys [5].
- Although effective for habit formation, Fabulous relies on static, predefined routines and does not adapt dynamically to real-time user behavior or changing workloads [5].
- Compared to Sproutly, Fabulous has a heavier onboarding and commitment structure, making it less suitable for users seeking lightweight productivity support.

### 2.2.4 RescueTime and Freedom

- RescueTime and Freedom focus on digital self-control through activity tracking and distraction blocking [8], [9].
- These tools can effectively limit access to distracting websites or apps but provide little in the way of reflective support, task assistance, or personalization.

- Their strict blocking approach may be counterproductive for users who prefer autonomy, as noted in DSCT research [3], [4].

These established tools each address a piece of the productivity problem but lack an integrated system that adapts dynamically to user needs, supports large or ambiguous tasks, and offers optional accountability without imposing coercive measures.

# 3. Proposed Software Architecture

## 3.1 Overview

Sproutly is a cross-platform mobile app that helps users stay focused during study or work sessions by tying consistency to a virtual companion. The core loop is simple: the user plans what they want to do, starts a focus session, follows the rules of their chosen coaching mode, and then earns progress (or receives penalties) based on how the session went. Proof of work is optional and, when enabled, can be used to strengthen the reward flow by asking the user to upload a photo at the end of a session for AI review and validation.

At a high level, the system follows a layered, modular structure. The Flutter mobile client handles the user experience and the focus session flow. It owns all of the UI rendering, local session state, timer logic, and platform-specific behavior such as app restriction enforcement and widget rendering. The client does not access the database or call AI services directly; every data request and every AI-dependent action goes through the backend over RESTful API calls via HTTPS. A backend layer built with FastAPI acts as the bridge between the app, the database, and AI components. The backend and database are both hosted on Railway. The backend provides RESTful endpoints based on domain (user-facing endpoints such as create an account, modify an account, create/retrieve tasks, modify/manage schedule, create/search for notes, record mood check-ins and session logs/the AI-facing endpoints, request task breakdowns, proof of work verification, and schedule image parsing, generating weekly reports). When the client sends a request that involves AI processing, the backend constructs the appropriate prompt or payload, forwards it to the external AI service, receives the response, validates and restructures it into the application's own data format, and returns the result to the client. The client never talks to AI services on its own; the backend owns that boundary entirely. Cloud MySQL stores relevant user information. The schema covers users, devices, personality profiles, tasks and subtasks, focus sessions, companions, schedules and schedule events, notes, distraction notes, mood check-ins, weekly reports, proof of work records, and blocked app configurations. The backend is the only layer with direct read and write access to the database, and all queries and mutations pass through FastAPI endpoint handlers that enforce validation, authorization, and data consistency before any database operation executes. The system uses Anthropic's Claude Sonnet 4.5 as the external AI service for AI-dependent features, including task breakdown into subtasks, proof of work photo and document verification, schedule image parsing via OCR, note summarization, and

mood-based task recommendation. AI responses are not passed through to the client raw; the backend parses and restructures them before returning anything to the mobile app.
Inside the mobile app, features are organized around clear responsibilities rather than one large, tangled block of logic. Core areas include task scheduling, timers (including work and break timing similar to Pomodoro), coaching modes that define strictness, an app restriction system, and progression systems like the companion's visual changes, streak tracking, milestones, and badges. Coaching modes range from Casual to Extreme, with differences in how quickly a user can override restrictions and how severe the penalties are when they break session rules. The intent is that users who want gentle support can get nudges, while users who want stronger accountability can choose modes that make quitting or overriding meaningfully costly.

App restriction is intentionally described as more than just a single "block everything" switch. The planned approach emphasizes nudges over strict blocking by default. When distracting apps are detected as open or running, the system can respond with notifications, and overrides are allowed for emergencies with a penalty. The requirements also acknowledge a platform split: Android supports stronger locking through usage access and overlay permissions, while iOS relies more on persistent reminders and harsher penalties because full background app locking is treated as not allowed in the project's current plan. The coaching mode selected by the user determines the severity of override penalties and the strength of restriction enforcement, and this configuration is stored server-side so that it stays consistent across devices.

Proof of work and AI support are handled through the backend layer. After a session, a user can submit a photo as evidence of completed work. The client uploads the media file to the backend, which holds it in temporary storage, forwards it to Claude Sonnet 4.5 for verification against the session's task description, and discards the media after the verification result is recorded. The backend can run the AI review step and return a result that the app uses to decide how the session should affect milestones, badges, and other rewards. The backend then writes the updated progression data back to the database, and the client receives the final result and renders the appropriate feedback screen. Separating AI work into the backend keeps the mobile app lighter and lets the team improve the verification approach without needing a full mobile release for every change. Finally, security and privacy is handled by Firebase auth.

## 3.2 Subsystem Decomposition

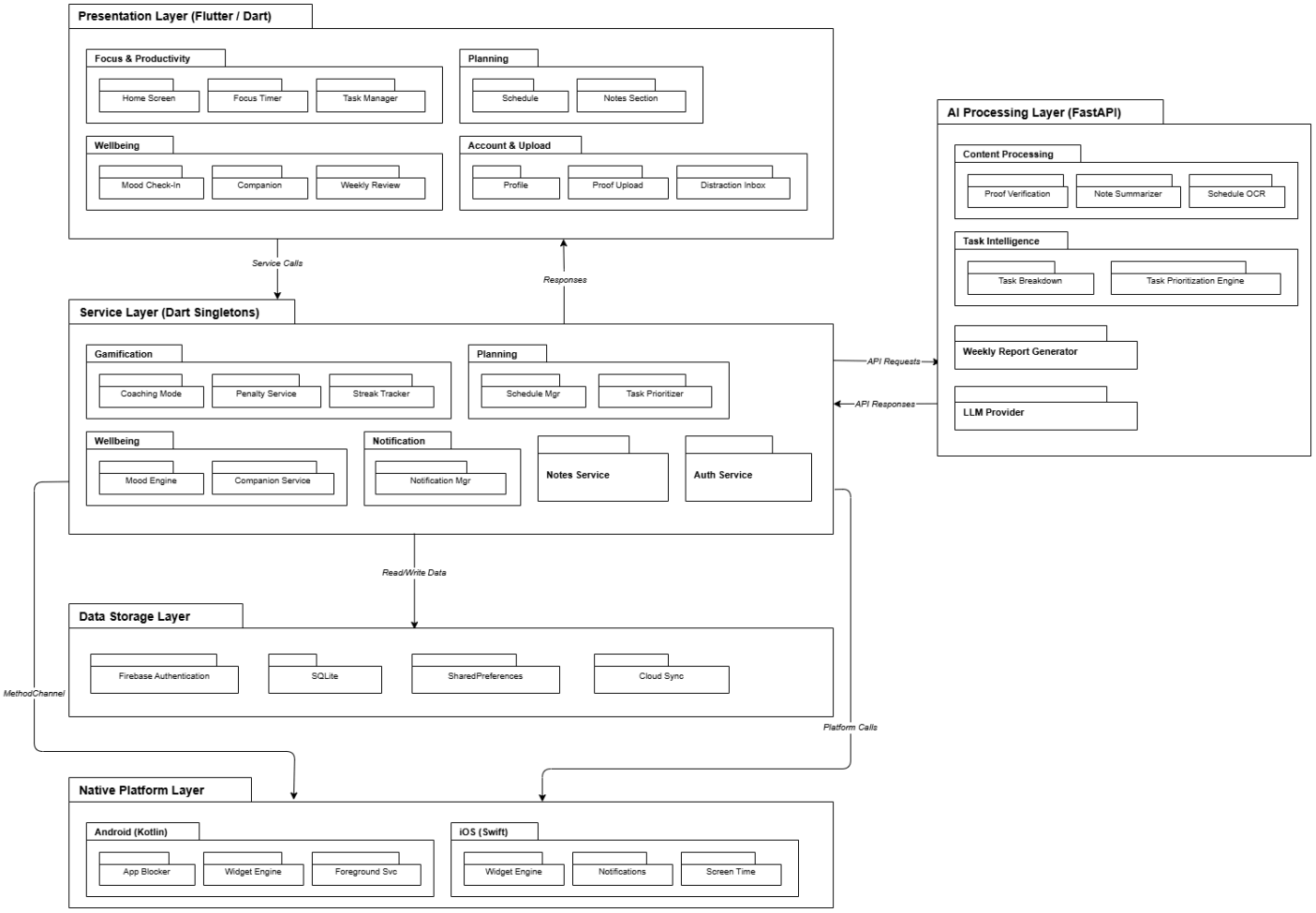


Figure 1: Subsystem Decomposition Diagram

### 3.2.1 Presentation Layer

**Purpose:** Provides the mobile user interface of Sproutly using Flutter. Enables users to route between screens, presents feedback and companion interactions and gets input from the user. The presentation layer is linked to backend and Firebase services.

**Subsystems:**

1. **Focus & Productivity:** Handles the main productivity flow including the dashboard screen which enables access to core modules, focus timer and task management.
2. **Planning:** Provides calendar based scheduling and note taking features to help users plan tasks and routines.
3. **Wellbeing:** Ensures the users' wellness with mood check-ins, companion features and weekly review. The companion module includes the screen to customize the companion as well.
4. **Account & Upload:** Handles account and profile related screens, proof upload for the Proof of Work feature and distraction inbox.

### 3.2.2 Service Layer

**Purpose:** Provide the necessary utility for certain features as singletons. These features include gamification elements, planner features, well-being related modules; notification, notes and authentication services.

1. **Gamification:** Coaching mode serves as a difficulty setting for the user. Penalty functionality creates negative incentives for certain coaching modes, scaling with the difficulty of the coaching mode selected. Streak tracking is used to track the user's consistency in using the app, to incentivize them to follow their schedules, habits, and plans.
2. **Planning:** Planning functionality includes one-time, non-recurring activities (such as deadlines for tasks or projects) as well as weekly schedules laid out in the calendar by the user.
3. **Well-Being:** Well-being functionality includes the user's self-reported data about their habits; such as mood, sleep, energy level, and more.
4. **Notifications:** Notifications services are used to provide notifications under certain app usage conditions. Notifications may be sent if the user is distracted when the app is running in the background, or if a timer has been paused for too long during an active focus session, and more. The frequency and content of notifications may differ based on the selected coaching mode.
5. **Notes Service:** Allows the user to take basic notes within the app.
6. **Auth Service:** Provides authentication services for user accounts. The service is handled on the Flutter client using Firebase Authentication. After authentication, newly created users can proceed and finalize their account setup, and existing users can sign in and out of the application.

### 3.2.3 AI Processing Layer

**Purpose:** Provides AI based functionality, including verifying proof uploads, summarizing notes, extracting schedules from images, assisting task planning, and generating weekly reports.

**Subsystems:**

1. **Content Processing:** Manages content that needs AI confirmation such as verifying proof uploaded by the user, summarizing user notes and extracting schedule information from uploaded images and adding them to the user schedule.
2. **Task Intelligence:** Supports task planning features such as task breakdown which divides tasks into smaller parts and task prioritization which highlights more urgent or higher priority tasks for the user.
3. **Weekly Report Generator:** Generates weekly reports including the user's total focus time, number of focus sessions, number of completed tasks, streak information, Proof of Work results and companion progress such as the XP gained. The generated report also includes reminders and suggestions from LLM.
4. **LLM Provider:** Manages requests to the Claude LLM that Sproutly uses for confirming the proof sent by the user for the Proof of Work part. This module enables communication with Claude LLM and returns its outputs to the service layer.

### 3.2.4 Data Storage Layer

**Purpose:** Provides storage for user option preferences, user account information and any other relevant data, such as tasks and schedules, which the user may be utilizing in the application. Interfacing with the data storage layer is done through the backend.
**Subsystems:**

1. **SQLite, SharedPreferences (Android) / UserDefaults (iOS):** Provides on-device, local storage for data. This data may include user settings, user preferences, and more. Certain data may be cached in local storage.
2. **Firebase:** Cloud storage used for account details, as well as task data. Primary account details include username, password, e-mail, or other unique personal information which may be used during sign-in. Usage of cloud storage provides a single source of truth for any relevant user information. Relevant user information may include task data, proof-of-work records, notes, weekly schedules, and more.
3. **Cloud Sync:** Provides the necessary implementation for the synchronization of data between local devices and data stored in the cloud.

### 3.2.5 Native Platform Layer

**Purpose:** Since certain features of the app can be implemented only on one specific OS, our app makes this distinction through the native platform layer. Each platform consists of its own native modules that are written in its own native language; in our case, this is Kotlin for Android and Swift for iOS. This layer is used to get access to these platform-specific features that can not be achieved with Flutter only. The native modules that each platform provides communicate with Flutter through a messaging system called the MethodChannel, which invokes native platform methods and receives the results.
**Subsystems:**

1. **Android (Kotlin):** This subsystem contains the following components: App Blocker, which blocks distracting apps from the user with some overriding and penalty conditions depending on the coaching mode during a focus session; Widget Engine responsible for rendering widgets in the home screen of the users phone displaying streak counts etc.; and Foreground Service, which maintains a persistent timer which will continue running even when the application is running in the background.
2. **iOS (Swift):** This subsystem contains the following components: Widget Engine, which behaves similarly to its Android counterpart, appearing on home screens and lock screens; Notifications Module, responsible for managing local notification scheduling; and lastly Screen Time, which uses Apple's Screen Time API to restrict app usage during focus sessions.

## Layer-to-Layer Communication

1. **Presentation Layer → Service Layer:** The presentation layer makes service calls to the service layer. These service calls are made to do the following operations: task creation, starting/stopping focus sessions, mood check-ins, companion messages, authentication, and schedule management.

2. **Service Layer → Presentation Layer:** From the service layer to the presentation layer, responses of these service calls are made. Some examples can be given as follows: updated state data, streak information, coaching feedback, companion responses, and processed schedule data.
3. **Service Layer → AI Processing Layer:** The service layer makes API requests to the AI processing layer. The API requests include: proof images for verification, notes for summarization, schedule images for OCR, and task lists for prioritization.
4. **AI Processing Layer → Service Layer:** The AI processing layer responds to the service layer with API responses including summarized content for the notes, extracted schedule data, and orderings of prioritized tasks.
5. **Service Layer → Data Storage Layer:** Service layer reads from and writes to data storage layer. Some examples can be given as retrieving tasks, notes, and schedules from SQLite; storing and reading user preferences, streak counts, and the coaching mode from SharedPreferences; managing authentication through firebase; synchronizing data across devices with Cloud Sync.
6. **Service Layer → Native Platform Layer:** Two types of communication happen between these two layers. The first one, MethodChannel, is used to invoke the native Android functionality such as app blocking, widget updates, and foreground service management. The second one, Platform Calls, invokes native iOS functionality like rendering widgets, scheduling notifications, as well as integrating Apple's Screen Time API.

## Advantages of the Architecture

1. **Layered Separation of Concerns:** The system has a layered design where each layer handles its own set of responsibilities. For example, there is a presentation layer, a service layer, a data storage layer, a native platform layer, and an AI processing layer. This improves maintainability and reduces coupling between components.
2. **Modularity:** The application is divided into modules, which simplify development, testing, and future maintenance. For example, there is a planning module, a gamification module, a wellbeing module, and a notification module.
3. **Cross-Platform Support:** The application is developed using Flutter, allowing it to support both Android and iOS. It also has access to platform-specific capabilities through its native platform layer, allowing it to use widgets, notifications, and app restriction capabilities provided by the native platforms.
4. **Centralized AI Processing:** All AI-related operations of the system are handled by the FastAPI backend. There is no AI-related operation handled directly by the mobile client. This improves security and makes it easier to integrate AI into the system, and also makes it easier to update it.
5. **Scalability and Extensibility:** Both AI-related components and backend services operate independently of each other. Hence, they can be scaled independently. This makes it easier to add new features to the system or add AI-related services to the system in the future.

6. **Security:** Authentication is done using Firebase Authentication, and the backend is the only layer that is allowed direct access to the database. This is to ensure that all operations are passed through validation and authorization checks.

## 3.3 Hardware/Software Mapping

The project is not built with, and does not depend on, any specific hardware components; therefore, this section is left blank.

## 3.4 Persistent Data Management

Sproutly uses both local and cloud-based storage to manage persistent data. On the mobile device, SharedPreferences is used to store small, local application data such as cached tasks, session information, and user settings and preferences, allowing the app to maintain quick access to frequently used data; whilst SQLite is used to store and access data which may appear in similar forms in large amounts, such as tasks and activities, which accumulate over time with app usage.

For long-term storage and cross-device synchronization, a cloud-hosted MySQL database stores core application data, where user authentication and identity management are handled through Firebase Authentication. The aforementioned data includes users, tasks and subtasks, focus sessions, schedules, notes, mood check-ins, and proof-of-work records. The mobile client does not access the database directly; instead, all read and write operations are handled through the FastAPI backend via secure REST API requests, which ensures proper validation, authorization, and data consistency.

## 3.5 Access Control and Security

Security and privacy are treated as baseline requirements rather than add-ons. The system includes Firebase for authentication and identity management. The auth system currently supports email and login via mail as part of the core requirements. Firebase provides excellent security for the identity layer, and hence, the team does not have to build and implement a custom auth system. Firebase issues authentication tokens that the mobile client includes in every API request to the backend, and the backend validates these tokens on each incoming request before processing it, ensuring that no unauthenticated or unauthorized access reaches the database or AI services. Firebase also provides the flexibility to enforce session tokens and access control across the mobile client and backend. Beyond that, the design aims to use encryption wherever possible, including encryption in transit for all client-backend communication and encryption at rest for sensitive user data in the database, and explicitly takes care of compliance goals set by privacy regulations such as KVKK, HIPAA, and GDPR.

# 4. Subsystem Services

## 4.1 Client Subsystem

Sproutly is a cross platform mobile application developed with Flutter. The client subsystem contains various screens that are important for the main user flows, including authentication, task management, focus sessions, companion related features, and user profile. Users begin with the sign in screen, new users can register from the sign up screen and complete the personality test before entering the main application. After successful authentication, the user is directed to the dashboard screen and can navigate between the Tasks, Calendar, Companion, and Profile screens. Some screens are accessible only from certain screens, such as customizing a companion through the Companion screen or starting a focus session through the timer setup screen. The overall navigation structure of the client subsystem is shown in the figure below.

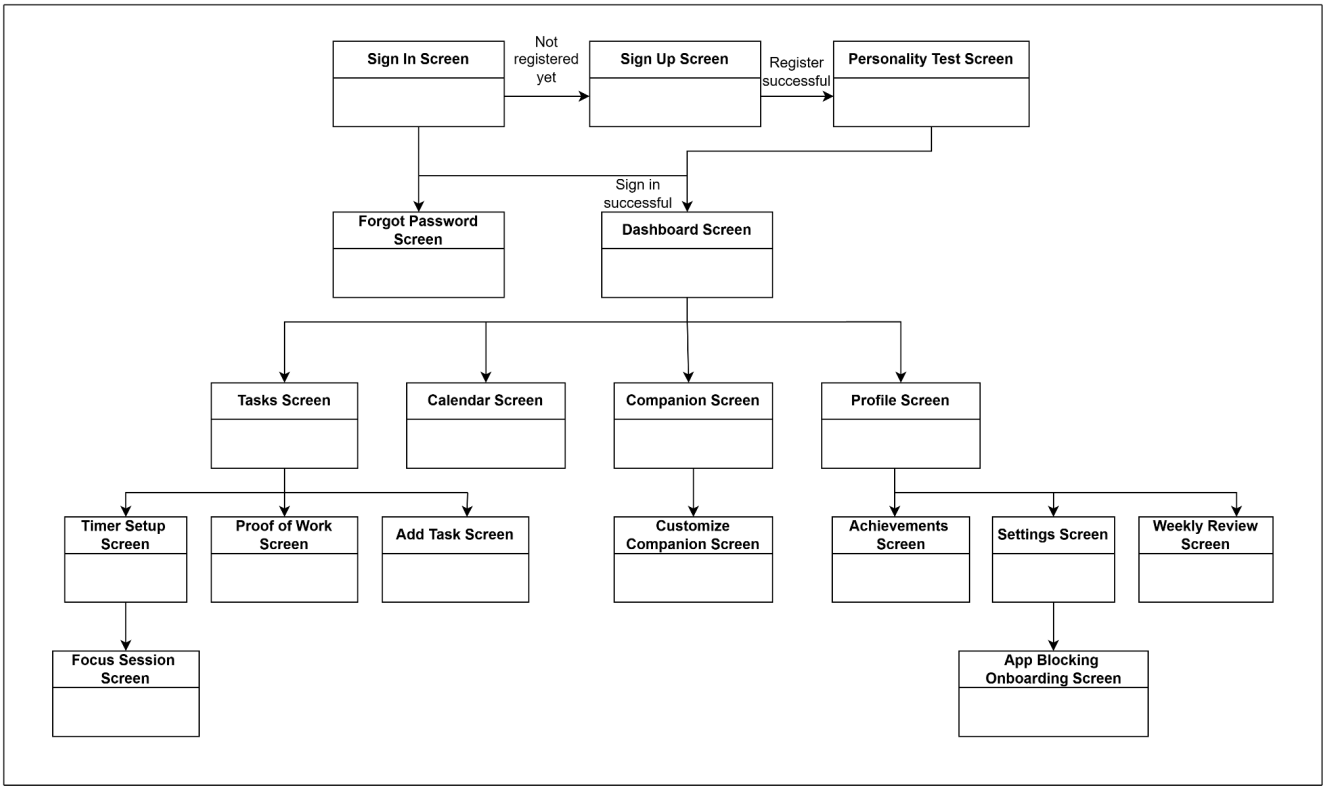


Figure 2: Client Subsystem Diagram

## 4.2 Backend Subsystem

The Backend Subsystem acts as the central communication layer between the mobile client, the database, and the AI components. Implemented using FastAPI, it exposes RESTful API endpoints that allow the client application to perform operations such as managing tasks, schedules, focus sessions, and user data. The backend validates requests using Firebase authentication tokens. It also handles all database interactions, storing and retrieving data

such as tasks, sessions, schedules, and companion progress while maintaining consistency and security. Additionally, the backend serves as a gateway for AI-related operations by forwarding relevant requests to the AI subsystem and returning processed results to the client.

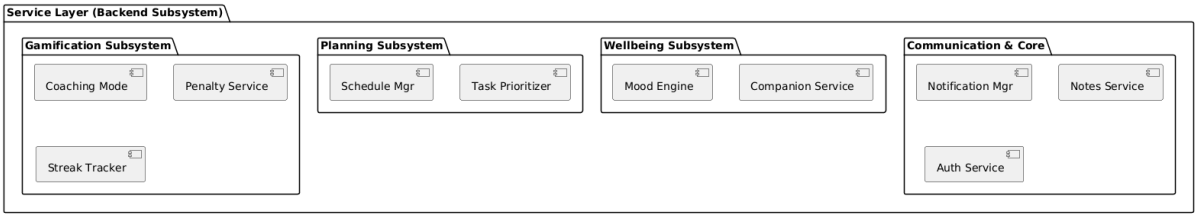


Figure 3: Backend Subsystem Diagram

## 4.3 AI Subsystem

The AI Subsystem provides intelligent assistance features that enhance the productivity support capabilities of our application. It operates within the backend environment and communicates with external LLM services to perform AI-based tasks. The subsystem supports functions such as breaking down complex tasks into smaller subtasks, verifying proof-of-work images uploaded by users, summarizing notes, and generating weekly productivity reports based on user activity data. It processes requests sent by the service layer, handles prompts for the LLM provider, and converts the responses into structured data that can be used by the application.

Figure 4: AI Subsystem Diagram

## 4.4 Object and Class Model

A general class diagram of the whole system is given below to give a better idea of the system. Due to its unreadability, in the following subsections, the class diagram of each domain in the system is shown, as well as the relationship between them.

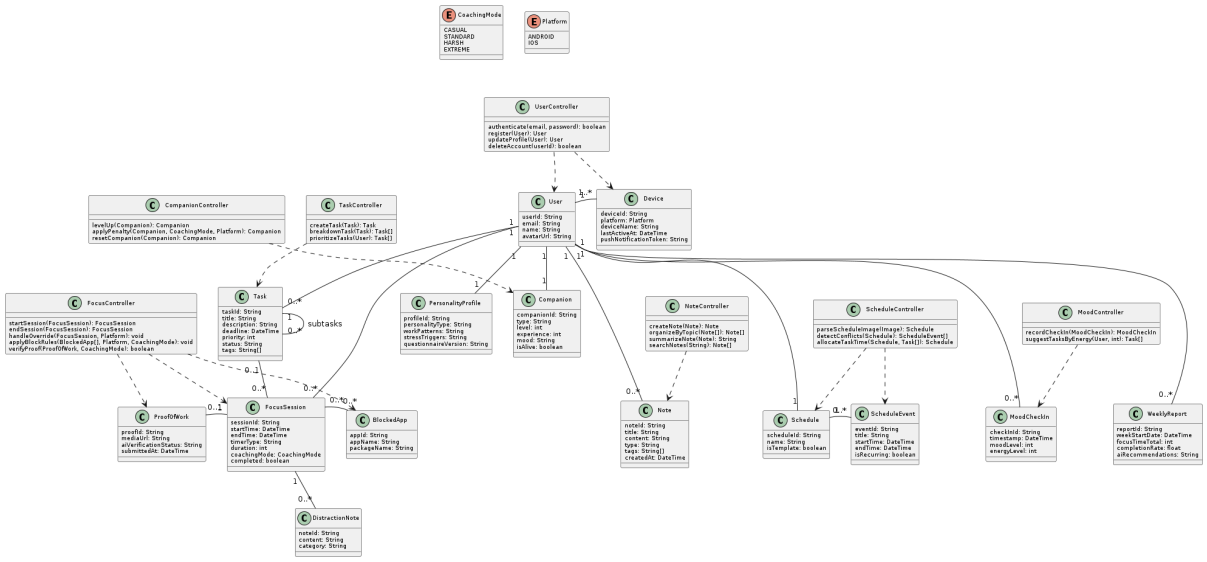


Figure 5: Sproutly Class Model

## 4.4.1 User Domain

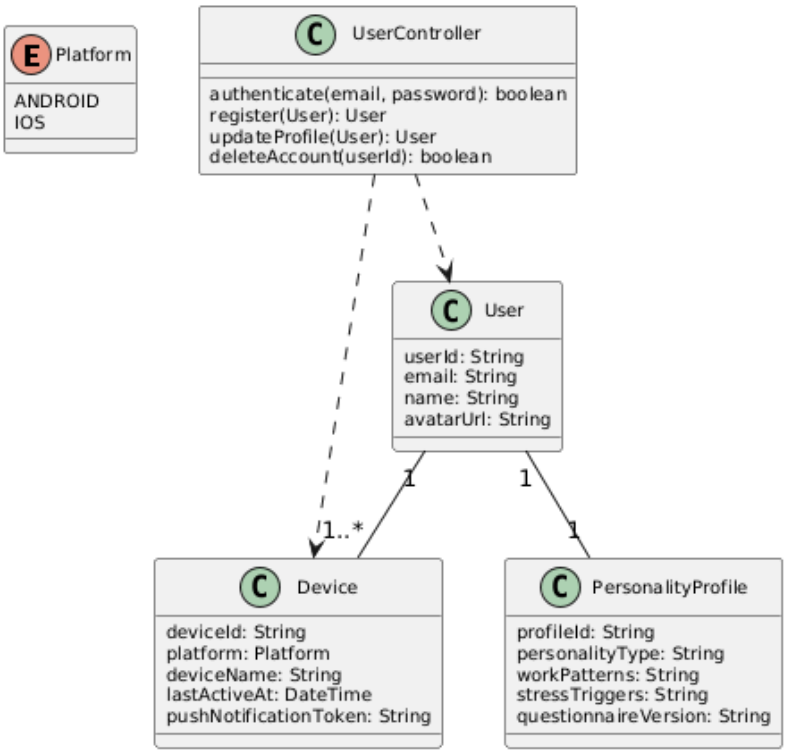


Figure 6: Class Diagram of User Domain

### 4.4.2 Companion Domain

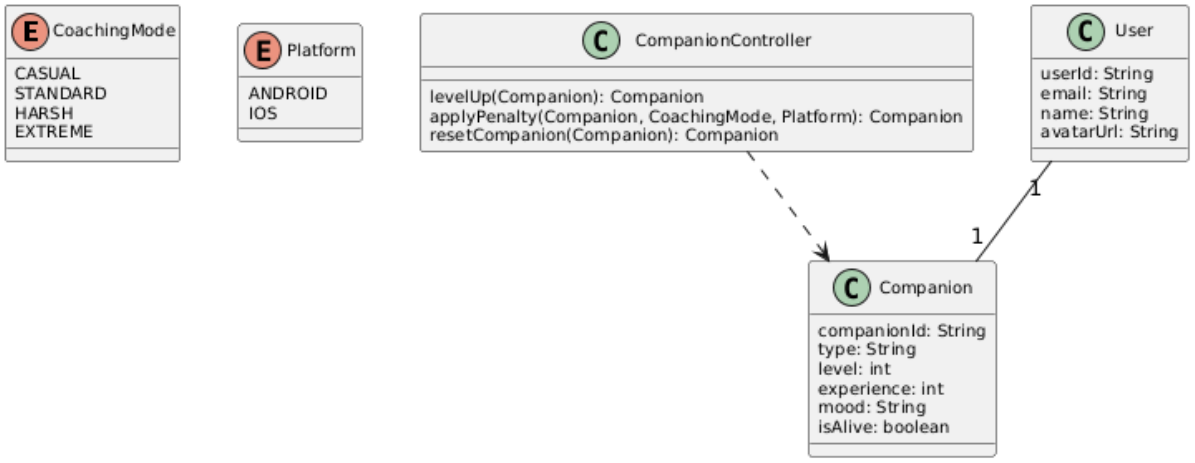


Figure 7: Class Diagram of Companion Domain

### 4.4.3 Task Domain

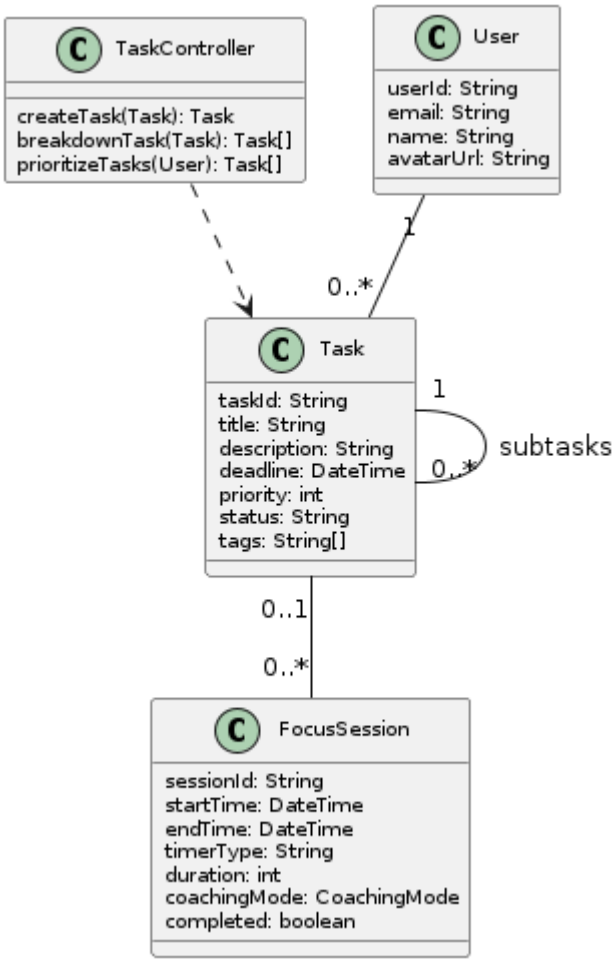


Figure 8: Class Diagram of Task Domain

### 4.4.4 Focus Domain

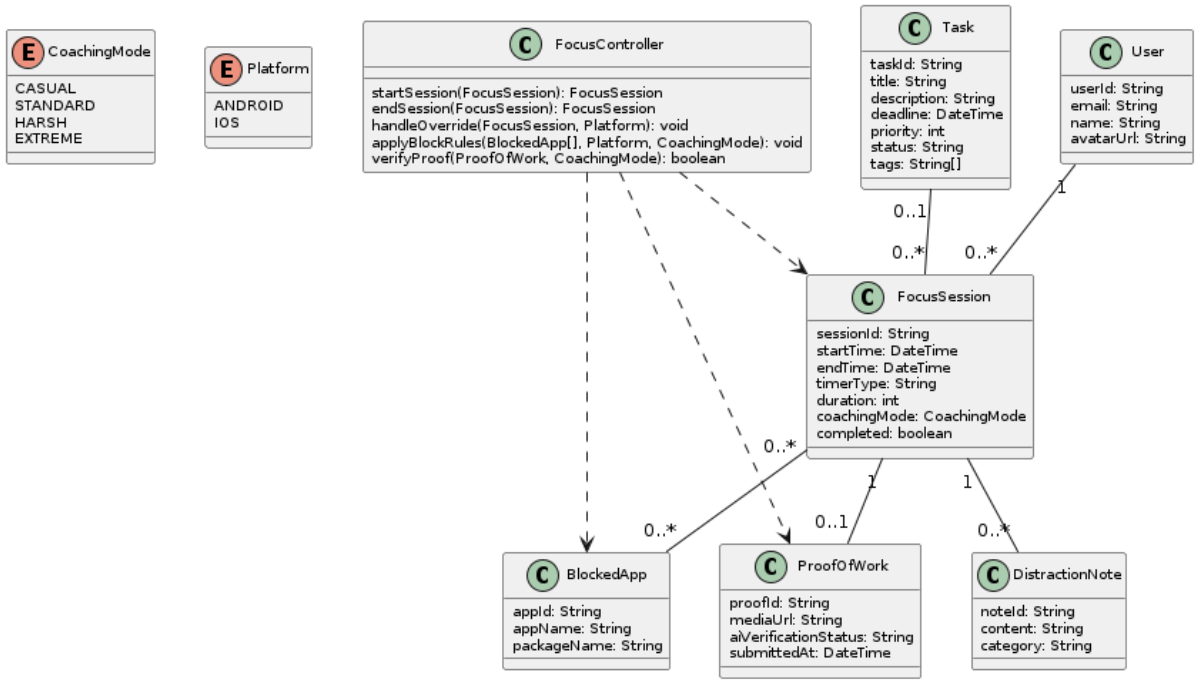


Figure 9: Class Diagram of Focus Domain

### 4.4.5 Notes Domain

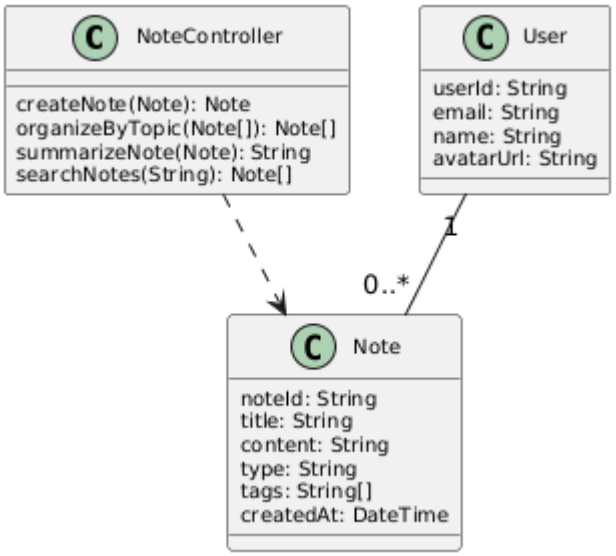


Figure 10: Class Diagram of Notes Domain

### 4.4.6 Schedule Domain

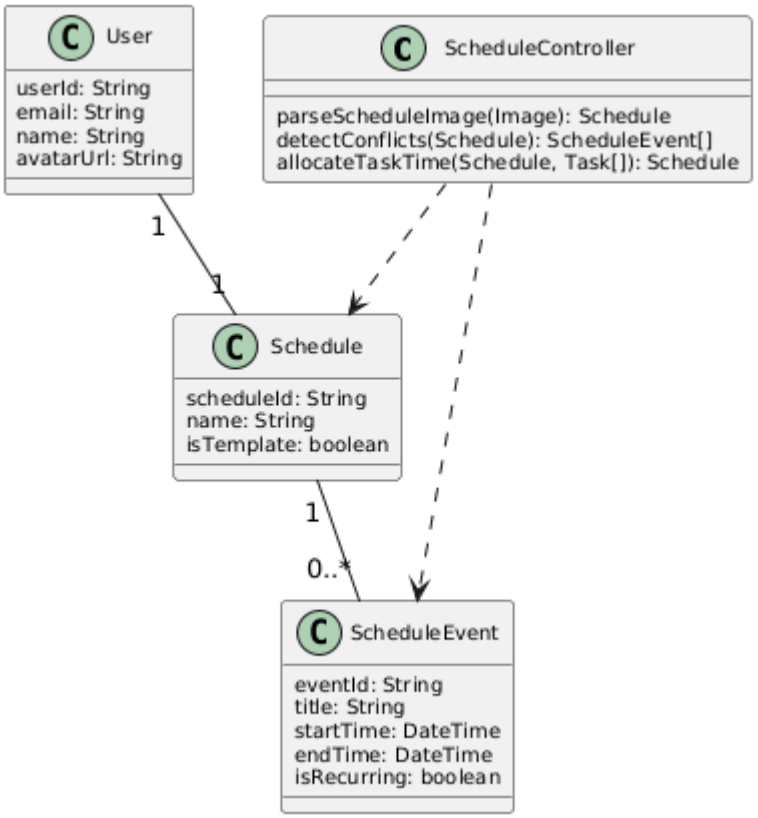


Figure 11: Class Diagram of Schedule Domain

### 4.4.7 Mood-tracking and Statistics Domain

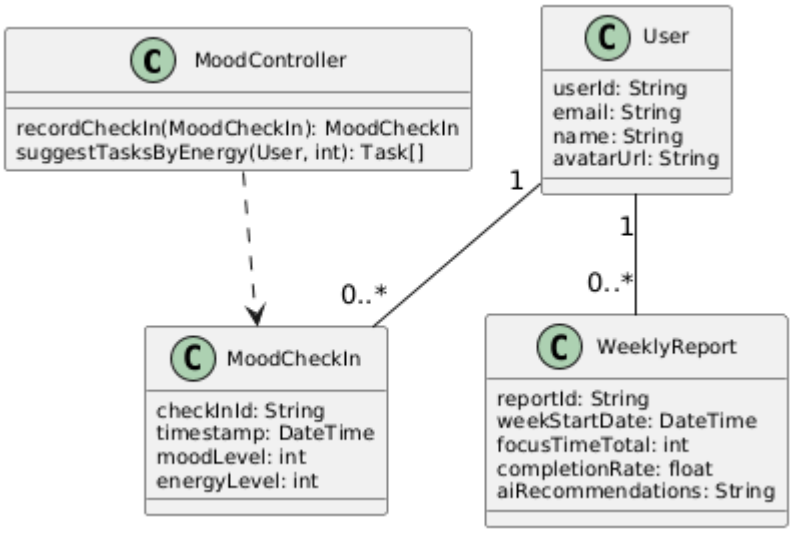


Figure 12: Class Diagram of Mood-tracking and Statistics Domain

# 5. Test Cases

**T-1: Non-Recurring Calendar Task Creation**

| Test ID | T-1 | Category | Functional |
|---|---|---|---|
| Objective | This test case is to verify that a user can successfully create a one-time, non-recurring task in the calendar without conflicts. | | |
| Steps | 1. Open the Sproutly mobile application and navigate to the Calendar/Scheduling module.<br>2. Click on the "Create New Task" option.<br>3. Enter a valid Task Name, a specific Date, and a Duration (e.g., "Study for CS465", October 20th, 2 hours).<br>4. Click 'Confirm' or 'Save' to create the task.<br>5. Verify the task appears on the specified date in the calendar view. | | |
| Expected | The task should be successfully created and displayed in the calendar. No error message should be triggered since there is no scheduling conflict. | | |

**T-2: AI Based Task Breakdown**

| Test ID | T-2 | Category | Functional |
|---|---|---|---|
| Objective | This test case is to verify that the AI module can successfully break down a large task into manageable subtasks. | | |
| Steps | 1. Navigate to the Tasks page and select the option to add a new task.<br>2. Enter a complex, long-duration task (e.g., "Finish Senior Design Report").<br>3. Select the "AI Breakdown" or "Split into subtasks" option.<br>4. Review the list of subtasks generated by the AI module.<br>5. Confirm and save the subtasks to the task list. | | |
| Expected | The AI should generate a clear, ordered list of smaller steps (subtasks) based on the description. These subtasks should be automatically added under the parent task. | | |

**T-3: Detection and Prevention of Overlapping One-Time Tasks**

| Test ID | T-3 | Category | Functional |
|---|---|---|---|
| Objective | This test case is to verify that the system detects and prevents overlapping tasks in the calendar. | | |
| Steps | 1. Navigate to the Calendar page where a task already exists for a specific | | |

| | |
|---|---|
| | time (e.g., 10:00 AM to 11:00 AM).<br>2. Attempt to create a new one-time task.<br>3. Set the time for the new task to overlap with the existing one (e.g., 10:30 AM to 11:30 AM).<br>4. Click 'Confirm' to save the new task. |
| Expected | The system should detect the conflict, prevent the task from being created or warn the user about the conflict so that the user can reschedule, and display an appropriate error message to the user. |

**T-4: Pomodoro Timer Input Validation**

| Test ID | T-4 | Category | Functional |
|---|---|---|---|
| Objective | Verify that the pomodoro timer can't be initialized with negative, empty, or extremely large values. | | |
| Steps | 1. Navigate to the Timer page after selecting a task.<br>2. Select 'Pomodoro Timer' as the timer of choice.<br>3. Set the fields to negative values, no value, or extremely large values.<br>4. Attempt to proceed to the next page. | | |
| Expected | The system should detect the invalid input, and block the user from going to the page after the timer initialization page. | | |

**T-5: Focus Session Reminders**

| Test ID | T-5 | Category | Functional |
|---|---|---|---|
| Objective | Verify that the application sends reminders when a focus session is ongoing. | | |
| Steps | 1. Start a focus session after selecting a task and setting up the timer.<br>2. Start the focus session timer.<br>3. Exit the application and open a different application whilst the timer is still running. Note that the app must still be open in the background. | | |
| Expected | After a specified amount of time, a notification should pop up reminding the user about their ongoing focus session. | | |

**T-6: Upcoming Task Reminders**

| Test ID | T-6 | Category | Functional |
|---|---|---|---|
| Objective | Verify that the application sends reminders for upcoming tasks. | | |

| | |
|---|---|
| Steps | 1. Navigate to the Calendar page and create a task.<br>2. Set the task's deadline to be 24 hours and 1 minute from now. |
| Expected | The system should automatically send a reminder for a task when that task has 24 hours remaining. |

**T-7: Proof of Work Upload Flow**

| | | | |
|---|---|---|---|
| Test ID | T-7 | Category | Functional |
| Objective | This test case is to verify the success path from completing a focus session through uploading a proof of work. | | |
| Steps | 1. Start a focus session in Standard Mode (1x XP).<br>2. Set the timer to run for 60 seconds.<br>3. Tap the "Complete Session" button.<br>4. Take a photo of the completed work.<br>5. Upload the proof for verification.<br>6. Read the verification result and session summary. | | |
| Expected | The proof upload screen opens. After submission, the verification result screen shows the AI confidence score. The session summary displays XP earned. Companion XP increases accordingly. | | |

**T-8: XP Multiplier Scaling**

| | | | |
|---|---|---|---|
| Test ID | T-8 | Category | Functional |
| Objective | This test case is to verify that the XP multiplier scales correctly among different coaching modes. | | |
| Steps | 1. Complete a 60-second session in Casual mode (0.8x). Note the XP earned.<br>2. Complete a 60-second session in Extreme mode (2.0x). Note the XP earned.<br>3. Compare the two values. | | |
| Expected | After an extreme mode session, the user should earn approximately 2.5 times more than a casual mode session, such a result will confirm the correctness of the XP multiplier. | | |

**T-9: Companion Evolution**

| | | | |
|---|---|---|---|
| Test ID | T-9 | Category | Functional |
| Objective | This test case is to verify that the companion evolves when the user completes a required number of tasks. | | |
| Steps | 1. Open the Sproutly application and navigate to the tasks screen.<br>2. Complete the required number of tasks needed for companion | | |

| | |
|---|---|
| | progression.<br>3. Open the companion screen from the home page.<br>4. Observe the companion's stage. |
| Expected | The companion should evolve to the next stage once the required number of tasks is completed, and the new stage should be displayed to the user. |

**T-10: Companion Mood Change**

| Test ID | T-10 | Category | Functional |
|---|---|---|---|
| Objective | This test case is to verify that the companion's mood changes based on user activity. | | |
| Steps | 1. Log in to the Sproutly application.<br>2. Complete several tasks or a focus session to trigger a positive interaction.<br>3. Navigate to the home and companion screens.<br>4. Observe the companion's mood and expression. | | |
| Expected | The companion's mood should update to a happy state after positive user activity. | | |

**T-11: Companion Progress Persistence**

| Test ID | T-11 | Category | Functional |
|---|---|---|---|
| Objective | This test case is to verify that the companion's progress is saved between sessions. | | |
| Steps | 1. Log in to the Sproutly application.<br>2. Complete tasks that affect companion progress.<br>3. Log out of the application.<br>4. Log back into the application.<br>5. Navigate to the companion screen. | | |
| Expected | The companion's stage, progress, and achievements should remain the same after logging back in. | | |

**T-12: Account Deletion**

| Test ID | T-12 | Category | Functional |
|---|---|---|---|
| Objective | This test case is to verify that a user can successfully delete their account. | | |
| Steps | 1. Open the Sproutly app and navigate to the Account Settings screen.<br>2. Select the "Delete Account" option.<br>3. Confirm the deletion request.<br>4. Attempt to log in again with the same credentials. | | |
| Expected | The user account should be permanently deleted, and the system should | | |

| | |
|---|---|
| | not allow login with the deleted credentials. |

**T-13: Account Creation**

| Test ID | T-13 | Category | Functional |
|---|---|---|---|
| Objective | This test case is to verify that a user can create a new account with valid information and access the application after successful registration. | | |
| Steps | 1. Open the Sproutly app and navigate to the sign up screen.<br>2. Enter valid registration information.<br>3. Tap the “Register” button.<br>4. Complete the “Personality Test” screen.<br>5. Confirm the app navigates to the dashboard screen after registration. | | |
| Expected | The account is created successfully, the personality test is completed, and the user is routed to the dashboard screen. | | |

**T-14: Successful Log In**

| Test ID | T-14 | Category | Functional |
|---|---|---|---|
| Objective | This test case is to verify that a registered user can log in with correct credentials and reach the dashboard screen. | | |
| Steps | 1. Open the Sproutly app and navigate to the sign in screen.<br>2. Enter a valid email and the correct password for an existing account.<br>3. Tap the “Sign In” button.<br>4. Confirm the app navigates to the dashboard screen after signing in. | | |
| Expected | Login succeeds and the user is directed to the dashboard. | | |

**T-15: Log In with Incorrect Password**

| Test ID | T-15 | Category | Functional |
|---|---|---|---|
| Objective | This test case is to verify that the system refuses invalid login attempts and shows a meaningful error message. | | |
| Steps | 1. Open the Sproutly app and navigate to the sign in screen.<br>2. Enter a valid email but an incorrect password.<br>3. Tap the “Sign In” button.<br>4. Observe the response shown by the application. | | |
| Expected | Login fails and the app displays a meaningful error message, no navigation occurs. | | |

**T-16: Recurring Calendar Task Creation**

| Test ID | T-16 | Category | Functional |
|---|---|---|---|

| | |
|---|---|
| Objective | This test case is to verify that  a user can create a weekly task with valid information when there is no scheduling conflict. |
| Steps | 1.  Navigate to the Calendar screen.<br>2. Select “Create new task.”<br>3. Enter a valid task name, select a day of week, set a start time and a duration.<br>4. Tap the “Save” button.<br>5. Verify the task appears on the calendar in the correct slot. |
| Expected | The recurring task is created successfully and displayed in the correct slot on the calendar with correct time and duration. |

**T-17: Calendar Task Input Validation**

| | | | |
|---|---|---|---|
| Test ID | T-17 | Category | Functional |
| Objective | This test case is to verify that the system does not allow creating a calendar task with invalid inputs. | | |
| Steps | 1. Navigate to the Calendar screen and create a new task.<br>2. Leave the task name empty or do not select a day or time.<br>3. Tap the “Save” button.<br>4. Observe the response shown by the application. | | |
| Expected | The task is not created and a meaningful error message is shown. | | |

**T-18: Edit Calendar Task**

| | | | |
|---|---|---|---|
| Test ID | T-18 | Category | Functional |
| Objective | This test case is to verify that the user can edit an existing calendar task and the changes are shown correctly in the calendar. | | |
| Steps | 1. Navigate to the Calendar screen and find an existing weekly task.<br>2. Tap the task to open its details.<br>3. Tap the “Edit” button.<br>4. Change a field to a valid non-conflicting value.<br>5. Tap the “Save” button.<br>6. Verify the calendar shows the updated version of that task. | | |
| Expected | The task is updated successfully and the calendar view shows the new schedule. | | |

**T-19: Pomodoro Timer Pause/Resume Consistency**

| | | | |
|---|---|---|---|
| Test ID | T-19 | Category | Functional |
| Objective | This test case is to verify that the Pomodoro countdown timer works without any time jumps during a start, pause or a resume operation. | | |

| | |
|---|---|
| Steps | 1. Start a Pomodoro session with any time interval you like (should be large enough to observe any possible time jump).<br>2. Tap the “Start” button to start the timer.<br>3. Observe the countdown for a few seconds for any unusual behaviour.<br>4. Tap the “Pause” button to stop the timer.<br>5. Wait a few seconds, then tap the “Resume” button.<br>6. Make sure that the timer continues from where it paused. |
| Expected | The timer counts down from the time interval picked by the user. The pausing action freezes the countdown; the resuming action makes the countdown continue from where it left off without any time jumps. |

**T-20: Start Focus Session**

| | | | |
|---|---|---|---|
| Test ID | T-20 | Category | Functional |
| Objective | This test case is to verify that a user is able to select an already created task from the task list to start a focus session after pressing the circular timer button at the bottom of the home screen. | | |
| Steps | 1. Log in and make sure at least one task exists in the system.<br>2. If no tasks exist, create one before starting a session.<br>3. Tap the circular timer button at the bottom of the home screen.<br>4. Tap on a task from the task list to select it.<br>5. Tap “Next” to proceed. | | |
| Expected | All of the active tasks can be viewed through the focus task screen. A user can select the one that they desire and the selected task will be highlighted. Tapping “Next” will navigate the user to the Timer Setup screen. | | |

**T-21: Successful Sign Out**

| | | | |
|---|---|---|---|
| Test ID | T-21 | Category | Functional |
| Objective | This test case is to verify that the user can sign out successfully and won’t be able to go back to the home page after signing out. | | |
| Steps | 1. Log in to the app.<br>2. Navigate to the Profile screen.<br>3. Press the “Sign Out” button.<br>4. Try to go back by pressing the back button or use the navigation methods provided by your OS. | | |
| Expected | The user is signed out of the application and redirected to the login screen. Tapping the back button does not redirect the user to the home page. | | |

**T-22: Session Quit in Casual Mode**

| | | | |
|---|---|---|---|
| Test ID | T-22 | Category | Functional |

| | |
|---|---|
| Objective | This test case is to verify that quitting a session without completion in Casual mode shows a simple confirmation dialog, and no XP penalties are applied. |
| Steps | 1. Note the companion's current XP value.<br>2. Start a focus session with Casual mode selected as the coaching mode.<br>3. Tap "Start" to start the timer.<br>4. Tap "Quit session" (before the countdown is over if Pomodoro is selected as the timer type).<br>5. Read the text inside the confirmation dialog.<br>6. Tap "Quit" to confirm it. |
| Expected | A confirmation dialog appears with the text "Are you sure you want to end this session?" with "Keep Focusing" and "Quit" as response buttons, as the user wants to quit the session. After the user confirms, no penalty dialog appears. The user is redirected to the home page where he/she can observe that the companion's XP remained the same. |

**T-23: Session Quit in Standart Mode**

| Test ID | T-23 | Category | Functional |
|---|---|---|---|
| Objective | This test case is to verify that quitting a session without completion in Standard mode shows a simple confirmation dialog, and -10 XP penalties are applied. | | |
| Steps | 1. Note the companion's current XP value.<br>2. Start a focus session with Standard mode selected as the coaching mode.<br>3. Tap "Start" to start the timer.<br>4. Tap "Quit session" (before the countdown is over if Pomodoro is selected as the timer type).<br>5. Read the warning message inside the confirmation dialog.<br>6. Tap "Quit" to confirm it.<br>7. Observe the penalty dialog and the final XP. | | |
| Expected | A confirmation dialog appears with the text "You will lose 10 XP if you quit now" with "Keep Focusing" and "Quit" as response buttons, as the user wants to quit the session. After the user confirms, a penalty dialog appears showing "-10 XP". The user is redirected to the home page where he/she can observe that the companion's XP is reduced by 10. | | |

**T-24: Focus Timer Background Persistence**

| Test ID | T-24 | Category | Non-functional |
|---|---|---|---|
| Objective | This test case is to verify that the timer still runs and the session state persists when the app goes to the background and returns. | | |

| | |
|---|---|
| Steps | 1. Start a focus session and start the timer.<br>2. Get out of the app without closing the app (it should run in the background).<br>3. Wait 10 seconds.<br>4. Reopen Sproutly.<br>5. Observe the timer state. |
| Expected | The timer continues running when the app is opened and returned to the foreground, the timer also accounts for the time elapsed while being in the background. The session is not lost or reset. |

**T-25: Prevention of Duplicate Navigation on Multiple Taps**

| | | | |
|---|---|---|---|
| Test ID | T-25 | Category | Non-functional |
| Objective | This test case is to verify that tapping the “Complete Session” button rapidly multiple times does not cause the app to navigate to multiple proof upload screens. | | |
| Steps | 1. Start a focus session and let the timer run for a few seconds.<br>2. Press the “Complete Session” button multiple times quickly.<br>3. Observe the navigations. | | |
| Expected | Only one proof upload screen appears, after returning from one of them another one does not appear. | | |

**T-26: Prevention of Duplicate Quit Confirmation Dialogs**

| | | | |
|---|---|---|---|
| Test ID | T-26 | Category | Non-functional |
| Objective | This test case is to verify that tapping the “Quit Session” button rapidly multiple times does not cause the app to show multiple stacked confirmation dialogs. | | |
| Steps | 1. Start a focus session.<br>2. Before the session ends, tap the “Quit Session” button multiple times quickly.<br>3. Observe the number of confirmation dialogs displayed. | | |
| Expected | Only one quit confirmation dialog appears. | | |

**T-27: Timer Pause on Quit Confirmation Dialog**

| | | | |
|---|---|---|---|
| Test ID | T-27 | Category | Non-functional |
| Objective | This test case is to verify that the timer pauses when the quit confirmation dialog appears on the tap of the “Quit Session” button, and does not account for the time spent while the dialog is open. | | |
| Steps | 1. Start a focus session with Pomodoro countdown selected as the timer | | |

| | |
|---|---|
| | choice.<br>2. Tap the “Quit Session” button and note the timer value right at the moment of pressing the button.<br>3. Wait 10 seconds while the confirmation dialog is open.<br>4. Tap “Keep Focusing” to dismiss the dialog.<br>5. Compare the timer value to the noted timer value. |
| Expected | The timer value after dismissing the dialog appears to be the same as the value before opening the quit confirmation dialog. The 10 seconds spent looking at the dialog are not counted. |

**T-28: Custom Pomodoro Time Configuration**

| Test ID | T-28 | Category | Functional |
|---|---|---|---|
| Objective | This test case is to verify that the Pomodoro timer can be configured with any interval that the user enters and that the timer works according to the new custom values. | | |
| Steps | 1. Start a focus session.<br>2. During the timer setup, pick the Pomodoro (countdown) timer option.<br>3. Change the focus, short break, and long break fields to any interval you would like.<br>4. Change the “Long break after” field as well.<br>5. Tap the “Start Session” button. | | |
| Expected | The new entered values are applied during the countdown. Values are saved to SharedPreferences for the next session. | | |

**T-29: No XP for Zero Duration Focus Session**

| Test ID | T-29 | Category | Functional |
|---|---|---|---|
| Objective | This test case is to verify that completing a focus session with zero focus time yields no XP no matter what the coaching mode and its XP multiplier is. | | |
| Steps | 1. Start a focus session in any coaching mode.<br>2. As soon as the timer starts, press the “Complete Session” button, not letting the timer run.<br>3. Upload the proof and get verified by AI.<br>4. Observe the XP earned in the session summary screen. | | |
| Expected | In the session summary, the screen shows that 0 XP is earned. This is valid for any coaching mode. | | |

**T-30: Automatic Switch from Focus time to Short Break Period**

| Test ID | T-30 | Category | Functional |
|---|---|---|---|

| | |
|---|---|
| Objective | This test case is to verify that the Pomodoro timer automatically switches to Short break time when the user's focus session time period ends. |
| Steps | 1. Start a focus session with Pomodoro (countdown) timer type selected.<br>2. Set the focus field to a value you would like keeping it minimal to not wait for a long time.<br>3. Tap "Start" and wait until the time period you entered reaches 00:00.<br>4. Observe the app's actions when the timer reaches 00:00. |
| Expected | When the timer expires in a focus session, the app automatically switches to Short break. |

**T-31: Cooldown system in Harsh and Extreme Modes**

| Test ID | T-31 | Category | Functional |
|---|---|---|---|
| Objective | This test case is to verify that starting a new focus session is not allowed right after quitting one in Harsh or Extreme mode due to the cooldown system being active. | | |
| Steps | 1. Activate the cooldown system by quitting a focus session in Harsh or Extreme mode.<br>2. While the cooldown banner is visible in the home screen, try to start a focus session by tapping the circular timer button at the bottom of the home screen.<br>3. Follow the starting a focus session process by selecting a task and configuring the timer settings.<br>4. Tap the "Start Session" button.<br>5. Read the feedback given on the screen. | | |
| Expected | A red error message appears at the bottom of the screen with the text "Focus sessions locked for X more days", and the user is not allowed to start a focus session, preventing the companion from evolving through sessions. | | |

**T-32: Blocking Overlays in Blocked Apps**

| Test ID | T-32 | Category | Functional |
|---|---|---|---|
| Objective | This test case is to verify that a blocking overlay is shown when the user tries to enter a blocked app during a focus session and that the return button to return from that screen works. (This test is only valid for Android users as this functionality is only available for them) | | |
| Steps | 1. Add any distracting app to your blocked apps list.<br>2. Grant all required permissions through the app blocking onboarding screen that can be reached from the profile page.<br>3. Start a session in Standard mode.<br>4. Switch to the app that you added to the blocked apps list.<br>5. Observe the blocking overlay that comes up. | | |

| | |
|---|---|
| | 6. Tap “Return to Focus”. |
| Expected | An overlay appears once the user tries to open an app that is present in the blocked apps list. This overlay makes sure that it is not allowed to enter the app at the moment with the companion’s image showing. Tapping the “Return to Focus” button returns the user back to Sproutly. |

**T-33: Personalized AI Recommendations**

| Test ID | T-33 | Category | Functional |
|---|---|---|---|
| Objective | This test case is to verify that the AI module gives study recommendations customized to the user based on their study patterns derived from the session histories. | | |
| Steps | 1. Complete at least 10 focus sessions across different tasks and times of the day, as well as across different days of the week.<br>2. Navigate to the Insights page.<br>3. Read the AI generated recommendations.<br>4. Make sure that the data used to provide the recommendations actually reference the user’s data. For instance, “You focus best during the evenings” or “You should consider using shorter sessions for Math related tasks”. | | |
| Expected | The AI module provides 2-3 recommendations based on the user’s study patterns. The suggestions are relevant and they are derived from the user's data. If there is not enough data to reach an outcome, a message is shown saying “More sessions are needed to give recommendations.” | | |

# 6. Consideration of Various Factors in Engineering Design

## 6.1 Constraints

This section discusses how public health, safety, security, welfare, and global, cultural, social, environmental, and economic factors influenced the project’s detailed design decisions.

### 6.1.1 Public Health

Sproutly interacts with users’ productivity habits and mental well being, which affected the design to position the system as a supportive productivity tool rather than a medical system. Therefore, public health considerations are a crucial part of the design, especially in how the system delivers feedback. For this reason, AI outputs are limited to productivity coaching and do not provide health related advice. Public health considerations also shaped the Proof of Work feature, which is designed to support healthy productivity behavior rather than increase stress or guilt so it is treated as an optional mechanism and the user can disable this feature when needed.

### 6.1.2 Public Safety

Safety considerations affected the design of the focus and app restriction features by prioritizing user control. The system includes emergency exits to ensure users maintain control over their devices at all times. The restriction feature is designed to let users choose a mode, so restrictions reflect user preferences rather than forcing a fixed behavior for every user. Restriction modes are designed with safe default settings, so the initial mode is the least disruptive option and users must explicitly choose a stricter mode if they want one. Actions that can significantly affect user experience, such as enabling a restrictive mode, are designed to require an explicit warning and user confirmation. In addition, the system also is designed to avoid permanent or irreversible actions that could negatively affect users or their devices. Since Sproutly is a cross-platform application and there are differences between Android and iOS, restriction features are designed to have alternatives for when an operating system doesn't support them.

### 6.1.3 Public Security

User access is protected through Firebase Authentication, and user sessions remain active even after the app is closed and reopened so users stay signed in across app restarts. Communication with Firebase services is handled over HTTPS/TLS to protect data while being transmitted. Data collection is limited to what is required for core functionality, such as tasks, focus session configuration, and basic progress information, and unnecessary sensitive personal data is not collected.

### 6.1.4 Public Welfare

User welfare strongly influenced the design of gamification and penalty related features. The system is designed to use respectful and supportive feedback, avoiding shaming language. Gamification and penalty mechanisms are kept simple and customizable, allowing users to adjust or disable features such as Proof of Work based on their preferences. In addition, negative outcomes are designed to offer recovery options, such as retry or alternative flows, so the system supports motivation rather than applying overly harsh penalties.

### 6.1.5 Global Factors

Global differences in working habits influenced the design of flexible coaching modes that users can choose from rather than enforcing a single productivity model. Also, global differences in daily schedules and time zones were considered during the design of the project, ensuring that the system supports flexible scheduling rather than fixed productivity timelines. In addition, the system supports user configurable schedules, allowing users to set their own preferred working hours.

### 6.1.6 Cultural Factors

Cultural differences in productivity, discipline, and gamification affected the design by focusing on neutral language throughout the interface and feedback messages. The system avoids culture specific or potentially offensive phrasing and presents motivation in an acceptable language. In addition, multiple virtual companions are included, allowing users to choose a preferred companion rather than limiting users to one fixed companion.

### 6.1.7 Social Factors

Differences in users' responsibilities and daily routines affected the design by keeping task management and scheduling adaptable rather than relying on a single fixed workflow. Focus sessions are also designed to support different modes and user configurable session durations so the use of focus sessions fits different working environments.

### 6.1.8 Environmental Factors

Environmental factors have a limited direct impact since Sproutly is a software based system. However, efficiency considerations affected the design by reducing unnecessary background processing and avoiding excessive cloud and AI usage. The system keeps AI features optional and user triggered so resource usage remains low during regular use.

### 6.1.9 Economic Factors

Budget limitations affected the design by shaping system scope, architecture and technology choices. To manage costs, AI features are designed to be optional and used in a controlled way, rather than being required for core functionality due to the use of paid AI APIs. In addition, the project uses a paid Rive license, so animation and companion work is prioritized to make efficient use of the available tooling budget. Overall budget considerations led to the use of open source tools and low cost technologies in the system architecture.

| | Effect level (0-10) | Effect |
|---|---|---|
| Public health | 8/10 | Affected the design to support mental well being by limiting AI to productivity coaching and keeping Proof of Work optional to avoid stress or guilt. |
| Public safety | 6/10 | The design ensures users can always exit restriction modes, and restrictions can be turned off or changed so users remain in control. |

| | | |
|---|---|---|
| Public security | 6/10 | Influenced the design by prioritizing authenticated access and encrypted communication to protect user data. |
| Public welfare | 9/10 | Significantly influenced the design by prioritizing supportive language, user controlled settings, and customizable gamification. |
| Global factors | 5/10 | Influenced the design to support flexibility for various ways of working and time zones. |
| Cultural factors | 4/10 | Affected the system to use neutral language and different virtual companions. |
| Social factors | 5/10 | Affected the design by enabling adaptable focus sessions and notifications for different daily routines. |
| Environmental factors | 3/10 | Had limited impact, but influenced avoidance of unnecessary resource usage and kept AI usage lightweight and optional. |
| Economic factors | 7/10 | Affected the design through controlled AI usage and cost conscious selection of tools and platforms. |

Table 1: Factors and Effect Levels

## 6.2 Standards

### 6.2.1 IEEE 830 Software Requirements Specification

IEEE 830 is referenced to keep the earlier requirements documentation clear, especially for functional and non-functional requirements. This consistent requirements style makes it easy to link design decisions back to specific requirements.

### 6.2.2 UML 2.5.1

System architecture and detailed design are documented using UML 2.5.1. In this report, UML diagrams are used to show the subsystem decomposition and the interaction flows between different subsystems and class and object diagrams.

### 6.2.3 ISO/IEC 25010

ISO/IEC 25010 is used as a reference to organize the project's non-functional requirements under standard software quality characteristics, such as usability, reliability, performance, maintainability, and scalability.

### 6.2.4 ISO/IEC/IEEE 12207

The overall project workflow is based on ISO/IEC/IEEE 12207 software life cycle principles, adapted to the scope of a student project. It follows a structured workflow across requirements, detailed design, implementation, and testing.

### 6.2.5 PEP 8

Python code written with FastAPI follows the PEP 8 style guide to maintain consistent formatting and readability. This improves maintainability by ensuring consistent coding style across the team and making future changes easier.

# 7. Teamwork Details

## 7.1 Contributing and functioning effectively on the team

- Weekly group meetings were conducted on the weekend, both online and in-person, to ensure team members are up-to-date on the status of the project and to ensure that responsibilities are distributed appropriately among team members.

- Both app development and report preparation workloads were distributed evenly among team members.

- Everyone was present and contributed to the demo and presentation.

## 7.2 Helping creating a collaborative and inclusive environment

To create a collaborative and inclusive environment, we followed an inclusive working style prioritizing transparency, responsibility, and mutual support. We implemented the following strategies to maintain this environment:

- We defined clear and achievable milestones during each meeting, and made sure that the workload was distributed fairly.

- Where possible, team members chose which features of the app, or sections of a report, they wanted to work on.

- We set realistic timelines and deadlines according to the difficulty of each task. For example, creating animations for companions or implementing bigger features requires more time compared to some other smaller features.

- Whenever an important decision needed to be made, all team members were consulted, and these decisions were made through consensus.

- We have centralized our communication and workflow with different tools. Zoom for simultaneous workloads and online meetings, WhatsApp for real-time updates, and GitHub for version management and collaborative development.

## 7.3 Taking lead role and sharing leadership on the team

Every member was assigned a different work package to lead from the start of the project, which ensured that everyone took the lead role and the team to share leadership.

- Kağan Rehber is leading the frontend development, ensuring a design that everyone agreed on and that was accessible.

- Bilge İdil Öziş is leading the AI/ML development, focusing on building robust models and integrating intelligent features into the system.

- Bilgehan Tuğcu is leading the backend development and database integration.

- Mennatallah Abouelenin is leading the companion system, creating companions and animations, and integrating the animated companions into the project.

- Elif Ercan is leading the gamification system, setting up XP, and handling achievements as well as companion designs.

# 8. Glossary

**Sproutly:** Name of the application in development. May be generically referred to as ‘app’ or ‘application’ throughout the document.
**Focus Session:** A certain duration during which a user actively focuses and does tasks.
Restriction Mode: The mode that the user chooses, depending on how intense they want the focus restrictions to be, to ensure the user stays focused.
**Companion:** The character designed to accompany, help, and motivate the user. Companions have animations and customizations.
**Streak:** The number of times in a row, in days, that the user has utilized the application.

**Distraction Inbox:** A section where users can write down distracting thoughts during a focus session so they can stay focused.
**Coaching Mode:** A selection of different modes with different intensity levels and focus features that users can choose based on their preferences.
**Personality Test:** A test that users answer at the beginning of their registration to set the user's initial preferences in the app.

# 9. References

[1] R. Garje, “Habit Tracking Apps Market Expected to Surpass USD 21 Billion by 2031,” *ResearchGate*, May 20, 2025.

[2] MarketIntelo, “Pomodoro Apps Market … 2025–2033,” *MarketIntelo*, 2025.

[3] A. Lyngs *et al*., “Self-Control in Cyberspace: Applying Dual Systems Theory to a Review of Digital Self-Control Tools,” *CHI ’19*, 2019.

[4] D. Biedermann, J. Schneider, and H. Drachsler, “Digital self-control interventions for distracting media multitasking: A systematic review,” *Education and Information Technologies*, 2024.

[5] The Fabulous, “The Science Behind Fabulous: Behavioral Research and Neuroscience,” Fabulous.

[6] Forest App, “Forest — Stay Focused, Be Present,” Forest App.

[7] Focus To-Do, “Focus To-Do — Pomodoro Timer & To-Do List,” Focus To-Do.

[8] RescueTime, “RescueTime: Time Tracking Software for Productivity,” RescueTime.

[9] Freedom, “Freedom — Block Distracting Websites, Apps & Games,” Freedom.